Davante

3

PROGRAMACIÓN

Programación estructurada

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Teorema de estructura 4

/ 3. Estructuras de selección 5

/ 4. Caso práctico 1: "Estructuras de control" 5

/ 5. Estructuras de selección simple: if 6

5.1. Anidación de selección simple: if-else 7

/ 6. Estructuras de selección múltiple: switch 9

/ 7. Caso práctico 2: "¿Anidamiento o condición múltiple?" 10

/ 8. Estructuras iterativas 10

8.1. Estructuras iterativas: while 11

8.2. Estructuras iterativas: do-while 12

8.3. Estructuras iterativas: for 13

/ 9. Salidas anticipadas 14

/ 10. Equivalencias entre bucles 15

/ 11. Resumen y resolución del caso práctico de la unidad 16

/ 12. Bibliografía 18

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Reconocer las estructuras de selección.

Conocer el concepto de anidación.

Conocer las estructuras de selección complejas.

Reconocer las estructuras de repetición.

Reconocer las estructuras de salto.

Saber cuándo utilizar la estructura de selección adecuada.

Saber cuándo utilizar la estructura de repetición adecuada.

/ 1. Introducción y contextualización práctica

Cuando desarrollamos componentes software, es necesario conocer todas las estructuras de control que tenemos a disposición. Es importante tener en cuenta que todos los lenguajes disponen de las mismas estructuras de control para desarrollar programas. Si bien es cierto que la sintaxis para realizar cada una de las tareas en los distintos lenguajes puede ser diferente.

Fig.1. Estructuras de control.

Conociendo las principales estructuras de control de flujo, seremos capaces de desarrollar algoritmos que se pueden traducir a cualquier lenguaje de programación, independientemente de la sintaxis del lenguaje.

En este tema, presentamos las principales estructuras de control de flujo que nos permitirán crear algoritmos más complejos.

A continuación, se va a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "¿Cine o parque?"

https://on.soundcloud.com/8pkXYobGPhrgjAU6A

/ 2. Teorema de estructura

Se denomina estructura a la manera en que diferentes partes se combinan para construir un todo. Un programa tendrá la estructura correspondiente a la **combinación de acciones individuales y cálculos parciales**. Por lo tanto, la programación estructurada plantea combinar operaciones sencillas y claras para construir otras más complejas.

Con anterioridad a los años 60, los programas consistían en un conjunto de instrucciones sin estructura ninguna, lo que dificultaba enormemente su mantenimiento. Para solucionar esa problemática, a finales de los años sesenta, Dijkstra, junto con otros autores, desarrollan los principios de la programación estructurada, estableciendo que un programa bien estructurado debía cumplir con los siguientes principios:

- El código debe ser entendible sin necesidad de información adicional.
- El programa podrá ser desarrollado por partes fácilmente ensamblables.
- Las diferentes partes del programa pueden ser modificadas o cambiadas, sin que esto afecte al resto del programa.

Un programa es estructurado si se expresa únicamente mediante combinaciones de las estructuras básicas: secuencial, alternativa e iterativa.

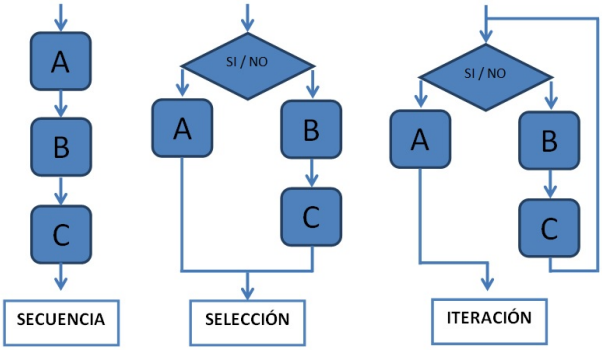

Fig.2. Estructuras básicas de la programación.

Con el teorema del programa estructurado se demostró que dado un programa P no estructurado con una única entrada y una única salida y sin interrupciones en el flujo de control, es posible construir un programa P' estructurado únicamente con estas tres estructuras básicas que obtenga los mismos resultados sin necesidad de usar el salto incondicional.

La programación estructurada aporta las siguientes ventajas respecto a las técnicas de programación anteriores:

- Los programas son más legibles o fáciles de entender y, por tanto, más fáciles de depurar y modificar.
- Reducción del esfuerzo en las pruebas.
- Reducción de los costes de mantenimiento.
- Aumento de la productividad del programador. Un programador que no haya realizado un código podrá fácilmente: entender su diseño y corregir posibles errores.

No obstante, tiene la siguiente desventaja:

- Se obtiene un único bloque de programa, el cual puede ser difícil de mantener si crece.

/ 3. Estructuras de selección

Durante el desarrollo de algoritmos y, por extensión, de soluciones software, es necesario poder tomar decisiones sobre el flujo de ejecución del programa, dependiendo de los resultados intermedios o de los parámetros y valores de entrada al propio programa. Vamos a definir el **flujo de ejecución** de un programa como **la secuencia de acciones o instrucciones** que tienen lugar para poder llevar a cabo **la implementación del algoritmo**. Podemos deducir que, el resultado puede ser diferente, dependiendo de la secuencia de acciones implementadas.

Durante el flujo de ejecución es posible poder tomar decisiones sobre qué secuencia de acciones llevar a cabo. De esta forma, estaríamos dotando al algoritmo de la capacidad de decisión sobre qué tareas realizar dependiendo de los valores de entrada. Existen diferentes estructuras que nos permiten tomar este tipo de decisiones para establecer bifurcaciones durante la ejecución del algoritmo. Normalmente, estás bifurcaciones están delimitadas por alguna condición que podemos analizar en los valores de las variables durante la ejecución de las instrucciones.

Gracias a las instrucciones de selección es posible romper el orden secuencial de ejecución. Hasta ahora hemos hablado de un orden en el cual las instrucciones se ejecutan una después de la otra, siendo obligatorio la ejecución de todas las instrucciones.

Con la introducción de las instrucciones de selección puede haber fragmentos de código que no se ejecuten.

Las instrucciones de selección sirven de base para implementar otras estructuras de control de flujo. Por ejemplo, durante las instrucciones de iteración es necesario poder comprobar diferentes condiciones para saber si seguimos ejecutando las iteraciones solicitadas en el algoritmo. Por este motivo, podemos decir que existen al menos tres estructuras de control en cualquier algoritmo, es más, podemos decir que con estas estructuras podemos construir cualquier algoritmo: **secuencia, selección y repetición**.

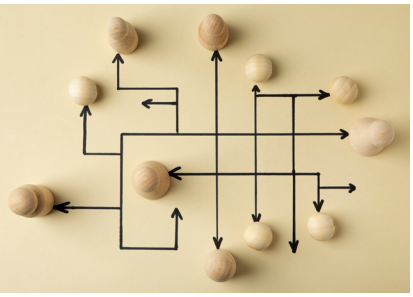

Fig.3. Flujo de selección.

/ 4. Caso práctico 1: "Estructuras de control"

Planteamiento: Ya se ha dicho que mediante el uso de las tres estructuras de control (secuencia, selección y repetición) se puede construir cualquier algoritmo, por lo que reconocer estas estructuras es fundamental para controlar el flujo de ejecución de un programa.

Nudo: Escribe un programa que solicite al usuario ingresar una serie de números enteros positivos. El programa debe calcular la suma de todos los números ingresados hasta que el usuario ingrese un número negativo. Una vez que se ingrese un número negativo, el programa debe mostrar la suma total de todos los números ingresados (excluyendo el número negativo) y terminar.

Justifique qué tipo de estructura de control se deberá emplear y realice su pseudocódigo.

Desenlace: Se debe emplear una estructura de repetición para que se repita hasta que se ingrese un número negativo y de selección para comprobar la condición de parada.

El pseudocódigo sería el siguiente:

```
Inicio del programa
        Inicializar suma a 0
        Repetir hasta que se ingrese un número negativo
                Solicitar al usuario ingresar un número entero positivo
                Si el número ingresado es negativo, salir del bucle
                Sumar el número ingresado a la variable suma
        Fin repetir
                Mostrar la suma total de todos los números
ingresados
Fin del programa
```

```
Inicio del programa
    Inicializar suma a 0
    Repetir hasta que se ingrese un número negativo
        Solicitar al usuario ingresar un número entero positivo
        Si el número ingresado es negativo, salir del bucle
        Sumar el número ingresado a la variable suma
    Fin repetir
        Mostrar la suma total de todos los números ingresados
Fin del programa
```

Fig.4. Resolución caso práctico.

/ 5. Estructuras de selección simple: if

La estructura de control de selección más simple que podemos encontrarnos es el **'si' condicional**. Normalmente, se utiliza junto a una expresión, de tal forma que si la expresión se cumple, se ejecuta el código que tenemos a continuación. La expresión que se utiliza en la condición tiene que devolver un **valor lógico**, es decir, verdadero o falso. En el caso de que no se cumpla la condición, el código que está dentro del bloque condicional no se ejecutará, continuando la ejecución del resto del programa. Se podría interpretar como un salto en el flujo de ejecución. De hecho, estas instrucciones también se conocen como saltos condicionales.

Supongamos el siguiente fragmento de ejemplo:

```
if (a == b){
  Sentencias si se cumple la condición
}
```

Fig.5. Código if.

Como ya se ha visto en temas anteriores, **es posible transformar un algoritmo en un diagrama de flujo.** De esta forma, también es posible transformar una expresión condicional en su correspondiente diagrama de flujo. En ocasiones, cuando el código aumenta de tamaño de manera significativa, cobra especial interés poder representar las condiciones, y el resto del flujo de ejecución, de una forma gráfica que permita al programador entender con facilidad el resto del algoritmo.

Normalmente, en un diagrama de flujo, una condición simple se representa mediante la siguiente figura:

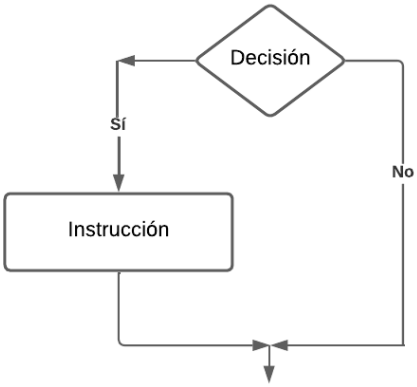

Fig.6. Ordinograma if.

Hasta ahora hemos visto una variante de la selección simple con una única parte, es decir, cuando se cumple la condición. Sin embargo, la variante más utilizada tiene dos partes: cuando se cumple la condición y cuando no se cumple la condición. De esta forma, se pueden establecer dos flujos diferentes de ejecución dependiendo si se satisface o no la primera condición. Además, es posible establecer una anidación de estas estructuras simples para que podamos tratar múltiples condiciones de manera lineal. En el caso de que no se cumpla ninguna, podemos seguir con la ejecución normal del programa.

¡! Sabías que...

Las estructuras de selección simple se traducen a código ensamblador, que implica un salto condicional. En ocasiones, los saltos en el flujo de ejecución pueden implicar una pérdida de rendimiento.

5.1. Anidación de selección simple: if-else

Como se ha comentado anteriormente, es posible realizar la anidación de diferentes estructuras de selección simple utilizando ambas partes de la selección. En el siguiente ejemplo se muestra una estructura de selección en la que podemos observar las dos partes de la expresión:

```
if (a == b){
        variable = Sentencias1 (cuando se cumple la condición)
} else
        variable = Sentencias2 (cuando no se cumple la condición)
}
```

Una forma más 'económica' y legible en el código sería sustituir el if-else por un **operador ternario**:

```
variable = a==b ? Sentencias1 : Sentencias2
```

Para realizar la anidación de las diferentes estructuras, es necesario utilizar ambas partes y combinarlas con sucesivas expresiones condicionales, que a su vez tienen también dos partes. Este flujo de ejecución suele utilizarse bastante en el desarrollo de algoritmos.

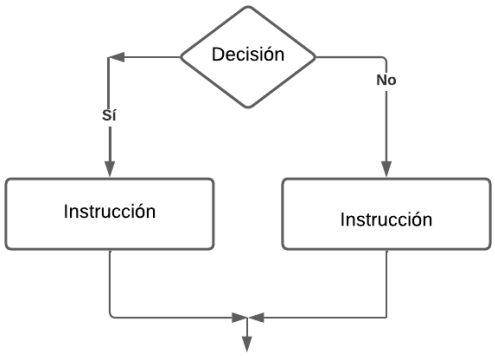

Fig.7. Ordinograma if-else.

Hay que tener en cuenta que, aunque se disponga de estas estructuras de control de flujos condicional, es altamente recomendable poder agrupar los casos de bifurcación de una manera genérica, es decir, que el número de bifurcaciones sea el mínimo posible en cada caso. Cuando se tienen múltiples anidaciones, se debe que considerar el orden de evaluación de cada una de las expresiones condicionales que tenemos en el algoritmo.

De esta forma, si la primera expresión de todas es verdadera, no se sigue con la evaluación del resto de expresiones. En el caso de que no se cumpla la primera expresión condicional, se seguirá el flujo de evaluación del resto de expresiones en el orden en el que se encuentran escritas hasta que se encuentre una verdadera. En el siguiente ejemplo se muestra una anidación de estructuras de selección simple:

```
if (a == b){
        if (c == d){
                Sentencias anidadas
        }
}
```

Como se puede observar, existen dos condiciones anidadas. Desde que se cumpla una, el resto no se ejecutará.

Recuerda...

Es posible anidar sentencias para tener mayor control sobre el flujo de ejecución de los algoritmos. Mediante la anidación de las sentencias es posible tener mayor control y detalle sobre las tareas a realizar en el algoritmo.

Audio 1. "Estructuras de selección y anidación"

https://on.soundcloud.com/VF59KMe5x1BiUYsS7

/ 6. Estructuras de selección múltiple: switch

Cuando un código dispone de múltiples caminos de ejecución y utilizamos estructuras de selección simples, podemos hacer que el código se vuelva ilegible o que su comprensión se vea afectada.

Para solucionar esto, podemos emplear estructuras de selección complejas que permiten simplificar, en gran medida, la escritura de sentencias de selección. Debemos tener en cuenta que estas estructuras complejas no introducen ninguna funcionalidad adicional a las vistas anteriormente. El único objetivo de estas estructuras es mejorar la comprensión y estructura general del código fuente. De esta forma, estaríamos garantizando la legibilidad del código para cualquier desarrollador que se enfrente a nuestro algoritmo en el futuro o, incluso, a nosotros mismos.

La sentencia más usada para utilizar estructuras de selección complejas es **switch** y su sintaxis junto al ordinograma es la siguiente:

```
switch expresión
  case valor-1
    instrucciones caso 1
  case valor-2
    instrucciones caso 2
  ...
  case {valores...}
    instrucciones caso k
  otherwise
    instrucciones
end
```

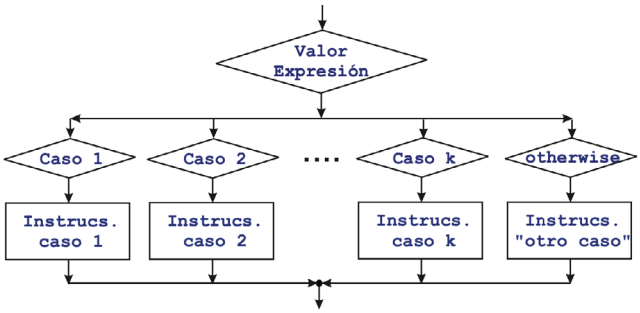

Fig.8. Sintaxis y ordinograma de la estructura switch.

Mediante esta estructura podemos establecer los diferentes casos que pueden tener lugar e indicar las acciones a llevar a cabo en cada uno de ellos. Dependiendo del lenguaje de programación, la condición de cada caso debe cumplir algunos requisitos. Así como en las estructuras de selección simple podemos tener una expresión relacional, en las estructuras complejas, los tipos de datos más ampliamente aceptados son carácter o entero.

En los lenguajes de programación más modernos se permiten utilizar otro tipo de datos como parte de la condición. A continuación, se muestra un ejemplo de sintaxis en el que se puede observar una estructura de selección compleja:

```
switch( numero )
{
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    case 4:
        System.out.println("Jueves");
        break;
    case 5:
        System.out.println("Viernes");
        break;
    case 6:
        System.out.println("Sábado");
        break;
    case 7:
        System.out.println("Domingo");
        break;
    default:
        System.out.println("Error. Día incorrecto.");
        break;
}
```

Fig.9. Ejemplo de bloque switch en Java.

En el ejemplo de la figura 9, el tipo de datos del selector debe ser entero (número), así como el valor de cada una de las etiquetas que forman cada uno de los casos.

/ 7. Caso práctico 2: "¿Anidamiento o condición múltiple?"

Planteamiento: La condición múltiple o anidamiento de condiciones en Java se refiere a la práctica de anidar varias instrucciones if dentro de otras instrucciones if. Esto permite evaluar múltiples condiciones en secuencia y ejecutar diferentes bloques de código según el resultado de estas condiciones.

Nudo: Pilar y José están realizando un ejercicio que consiste en comprobar que un usuario y una contraseña sean correctos, es decir, una validación. Para ello, Pilar cree que la mejor forma de hacerlo es con dos if anidados, cada uno con su propia condición, mientras que José piensa que sería más fácil hacerlo con un único if y una condición múltiple. ¿Qué piensas al respecto? ¿Crees que tiene razón Pilar o que la razón la tiene José? ¿Podrían tener razón los dos? ¿Acaso hay más de una forma de realizar el ejercicio?

Desenlace: Cuando en un programa necesitemos comprobar más de una condición, como es el caso del ejercicio de validación de usuario y contraseña que están realizando Pilar y José, no existe una manea única de hacerlo, sino que existen varias formas, así que, respondiendo la pregunta, los dos tienen razón, la comprobación se podrá hacer tanto con dos if anidados, uno para comprobar si el usuario es correcto y otro para la contraseña, o con un único if con una condición múltiple unida mediante el operador lógico AND, ya que tienen que ser verdad las dos validaciones.

La siguiente figura muestra la resolución con ambas propuestas:

```
//if anidados
if (usuario.equals("valorUsuario")) {
    if (usuario.equals("valorUsuario")) {
        System.out.println("Estas dentro del sistema");
    }

}

//condición múltiple
if (usuario.equals("valorUsuario") && contrasena.equals("valorContraseña") ){
    System.out.println("Estas dentro del sistema");
}
```

Fig.10. Resolución del caso práctico.

/ 8. Estructuras iterativas

Las estructuras iterativas nos permiten **realizar una secuencia de acciones un número determinado de veces**. Normalmente, para definir una estructura iterativa, hay que definir los siguientes elementos:

- **Cuerpo de la iteración**: Hace referencia al conjunto de sentencias y expresiones que van a formar parte de la iteración. Este bloque de sentencias es el que se va a repetir un número de veces.

- **Número de repeticiones**: Permiten indicar cuántas veces hay que repetir el cuerpo de la iteración.

- **Condición de finalización**: Es importante definir correctamente la condición por la cual la repetición o iteración finaliza. Dependiendo de la estructura de repetición que estemos utilizando, tendremos que plantear la condición de finalización de una forma u otra.

Un error muy frecuente en las estructuras iterativas es olvidar de establecer correctamente la condición de finalización. Cuando esto ocurre se puede tener una iteración infinita conocida como bucle infinito. Aunque esto parece una obviedad, dependiendo del algoritmo que se esté implementando, se puede caer con mayor o menor facilidad en un bucle infinito sin ser conscientes de ello. Cuando esto se une con sentencias de decisión, simples o complejas, es más complicado comprobar que el código está libre de este tipo de errores.

Básicamente, se puede decir que existen tres tipos de estructuras iterativas básicas. Es responsabilidad del programador elegir la estructura que más se adapte al problema a solucionar en cada momento. Se puede deducir que cada estructura iterativa encaja mejor en un tipo de problema que otra. De esta forma, se pueden distinguir las siguientes estructuras iterativas:

- **Evaluación de condición inicial**: Se trata de estructuras iterativas que realizan una comprobación de la condición de finalización antes de comenzar
- **Evaluación de condición final**: Estas estructuras realizan la comprobación de la condición al final.
- **Con contador**: Permiten garantizar un número determinado de iteraciones comprobando la condición de finalización en cada iteración.

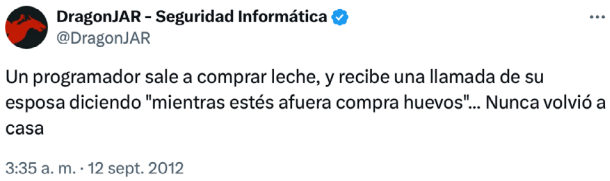

Fig.11. Estructuras iterativas.

8.1. Estructuras iterativas: while

La estructura iterativa **while** permite realizar una evaluación de la condición de finalización antes de comenzar la iteración. Esta característica permite que podamos estar seguros de que la condición es cierta antes de comenzar. La idea general de esta estructura es repetir el bloque de iteración mientras se cumpla la condición inicial. Al finalizar cada iteración, se vuelve a comprobar la condición, y en caso de seguir cumpliéndose, se realiza otra iteración.

Es importante tener en cuenta la condición de finalización que se está utilizando para garantizar que no se produzca un bucle infinito. Una forma de asegurarnos de que la condición puede cambiar en algún momento es utilizando una variable que sea modificada dentro de cada iteración como parte de la condición de finalización. Si por algún motivo la condición de la finalización permanece inalterada, con mucha probabilidad caeremos en un bucle infinito, influyendo negativamente en el rendimiento del programa.

```
...
while expresión-lógica
   instrucciones
end
...
```

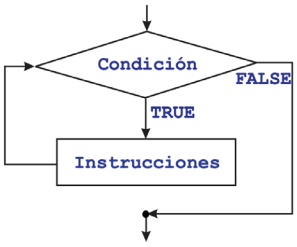

Fig.12. Sintaxis y ordinograma del bucle while.

De manera resumida, el funcionamiento de la instrucción **while** es el siguiente:

- Si la condición del bucle es cierta, se ejecuta el conjunto de instrucciones de la iteración. Estas sentencias se van a ejecutar en un orden secuencial.
- Si la condición del bucle es falsa, se salta a la siguiente instrucción que esté fuera del cuerpo de la iteración.

A continuación, se muestra un ejemplo de sentencia iterativa utilizando el bucle while:

```
int contador = 1;
while( contador <= 50 )
{
    System.out.println("El número es: " + contador);
    contador++;
}
```

Fig.13. Ejemplo de bucle while en Java.

Como se puede observar en el ejemplo anterior, se realizan tantas interacciones como veces se cumple la condición inicial.

8.2. Estructuras iterativas: do-while

En ocasiones, podemos necesitar que la condición que controla la iteración dentro del bucle no esté al comienzo, sino al final. De esta forma, nos aseguramos de que el bloque de instrucciones, que formarían parte de la iteración, se ejecuta al menos una vez. Como hemos dicho anteriormente, la condición que evalúa si repetimos o no el bloque de sentencias está al final del bloque de iteración.

En la mayoría de los lenguajes de programación se utiliza la sentencia **do-while** para establecer este tipo de estructuras. A continuación, se muestra un ejemplo de iteración empleando una evaluación al final del bloque de iteración, siendo el ordinograma de esta estructura el mostrado en la siguiente imagen:

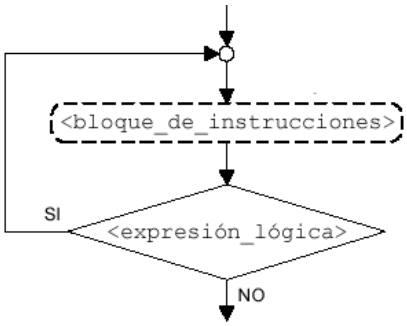

Fig.14. Ordinograma del bucle do-while.

El funcionamiento del bucle, por tanto, sería el siguiente:

1. Entra al bucle y ejecuta el bloque de código.
2. Al terminar la ejecución del bloque, comprueba si la condición es verdadera o falsa.
    a. Si la condición es verdadera, vuelve al punto 1.
    b. Si la condición es falsa, no entra al bucle y ejecuta la siguiente instrucción.

A continuación, se muestra un ejemplo en código:

```
int contador = 1;
do
{
    System.out.println("El número es: " + contador);
    contador++;
} while( contador <= 50 );
```

Fig.15. Ejemplo de bucle do-while en Java.

En el ejemplo se muestra un bucle en el que al menos se ejecuta una vez el bloque de instrucciones. A partir de este momento, se realiza la comprobación de la condición de finalización, es decir, (contador<50). Mientras se cumpla esta condición, se ejecutará el bloque de iteración. Dentro del bloque de iteración se imprimirá por pantalla el valor de la variable y, posteriormente, se incrementará en uno mediante la instrucción **contador++**. Cuando la variable que controla el bucle sea =>50, ya no se cumplirá la condición y finalizará la iteración.

Desde este momento se continuará con el resto del código a continuación de la iteración.

8.3. Estructuras iterativas: for

En ocasiones, interesa realizar un algoritmo que tiene una condición de iteración mediante un número de iteraciones determinado. Aunque este número de interacciones sea determinado, puede venir fijado por una variable que puede calcularse en tiempo de ejecución. De esta forma, podemos tener una iteración fijada por un valor numérico que se calcula de manera dinámica.

Recuerda...

Las instrucciones de control de flujo iterativo permiten realizar la misma tarea durante un número determinado de veces. Es importante fijar una condición de salida correcta para evitar caer en bucles infinitos.

Para utilizar este tipo de interacción, la mayoría de los lenguajes de programación introduce en la sentencia **for**. Gracias a esta sentencia se puede repetir un bloque de instrucciones un número determinado de veces. La siguiente figura muestra el ordinograma de este tipo de sentencia.

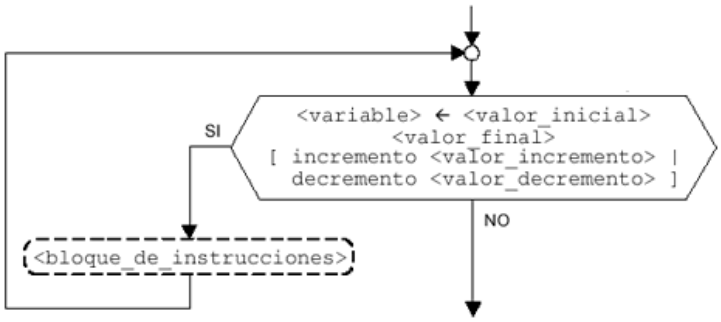

Fig.16. Ordinograma del bucle for.

La sintaxis para utilizar este tipo de bucle puede resultar algo confusa. A continuación, se muestra el esquema sintáctico de este tipo de sentencias iterativas (figura 17):

```
for (inicializacion; condicion-bucle; paso) {
sentencia 1
...
sentencia n
}
```

Fig.17. Código for.

Como se puede observar en el ejemplo sintáctico anterior, la sintaxis de este tipo de bucle es algo peculiar. A continuación, se explica en cada uno de los elementos que forman parte de la definición del bucle:

- **Inicialización**: Este campo permite inicializar las variables que van a controlar la ejecución del bucle. Normalmente, se le asigna un valor inicial que va a ir cambiando a lo largo de la ejecución del bucle.

- **Condición del bucle**: Es una expresión lógica que permite determinar cuántas iteraciones se van a realizar durante la ejecución del bucle. El bucle se va a repetir tantas veces como la condición sea verdadera.

- **Paso**: Indica cómo se va a modificar la variable que controla la condición del bucle. El paso puede ser positivo o negativo. Se puede deducir que el bucle puede ir incrementando o decrementando el contador que lo controla.

Vídeo 1. "Tabla de multiplicar con bucle for"

https://bit.ly/2U6vV4D

/ 9. Salidas anticipadas

Las sentencias de salto en programación permiten pasar el control de ejecución de un punto a otro del programa de forma incondicional.

Su uso es totalmente desaconsejable, salvo muy ciertas ocasiones. Por ejemplo, en Java, en las sentencias switch al terminar un caso, tendremos que utilizar la sentencia de salto break obligatoriamente, ya que así lo dice la sintaxis del lenguaje.

La razón de no aconsejar su uso en programas de hoy en día es debido a que provocan una mala estructuración del código fuente, incrementando la complejidad del mismo, ya que no es sencillo seguir el flujo del programa, y, por lo tanto, dificulta su depuración.

Existen varias instrucciones de salto, como son:

- **Instrucción goto**: Nos permite transferir el control de ejecución del programa a otro punto identificado por una etiqueta previamente creada. Esta instrucción provocaba que los programas fueran un auténtico caos, haciendo que la modificación de los mismos para mejorarlos fuera prácticamente imposible.

- **Instrucción break**: Utilizada para salir de un condicional o de un bucle de forma abrupta, pudiendo provocar fallos en los programas por el hecho de usarla incorrectamente. En Java la podemos utilizar, pero solo debemos hacerlo en las instrucciones switch.

- **Instrucción continue**: Se emplea en los bucles y su funcionamiento consiste en hacer que el bucle pase a la siguiente vuelta directamente, aumentando así su contador.

- **Instrucción return**: Se usa en los procedimientos que veremos más adelante, y su funcionamiento consiste en devolver el control de ejecución al punto donde se llamó al procedimiento.

En Java existen break, continue y return. Esta última sirve para devolver valores al volver de la invocación de funciones, mientras que las otras dos rompen el flujo de los bucles, como se ilustra en la figura:

```
// Ejemplo A: continue
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        continue;
    }
    System.out.println(i);
}

// Ejemplo B: break
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        break;
    }
    System.out.println(i);
}
```

Fig.18. Sentencias de salto en Java.

Vídeo 2. "Parando un bucle de forma correcta"

https://bit.ly/36WGUms

/ 10. Equivalencias entre bucles

Hemos visto que tenemos tres tipos de bucles: el bucle **while**, el **do-while** y el **for**, pero si vamos a realizar un programa donde necesitemos uno o varios bucles, ¿cuál elegimos?

La respuesta es sencilla, podemos utilizar el que queramos, ya que los bucles son equivalentes entre ellos, esto quiere decir que **cualquier ejercicio que contenga uno o varios bucles** podrá realizarse tanto con while como con do-while o con for.

Evidentemente, habrá que hacer algunos cambios si utilizamos un bucle u otro, como es lógico, pero, en la práctica, el resultado que obtengamos debería ser el mismo. Esto conlleva que podamos elegir un bucle y trabajar siempre con él. Un desarrollador en concreto puede sentirse más cómodo con el bucle while, por ejemplo, y puede decidir usarlo siempre como primera opción. El hecho de que podamos elegir, nos permitirá decidirnos por uno u otro, ya que nos encontraremos problemas donde será más sencilla una implementación con un bucle determinado que con los demás.

Esto se aprenderá con la práctica, pero las siguientes ideas nos servirán a modo orientativo:

- **For**: Cuando tengamos un número de iteraciones conocidas, por ejemplo, sumar los 100 primeros números enteros.
- **While**: Cuando el número de iteraciones sea desconocido porque dependa de una condición, por ejemplo, solicitar números al usuario hasta que sumen un valor determinado. No tendría sentido utilizar un for porque no sabemos cuántos números tendremos que leer.
- **Do-while**: Cuando se tenga que ejecutar al menos una vez, por ejemplo, solicitar números al usuario hasta que introduzca el 0. Puede que lo introduzca a la primera o a la cuarta vez, pero tendrá que introducir al menos uno.

Cuando tengamos algo más de experiencia, veremos que hay casos que se prestan a resolverse muchísimo más fácil con un bucle determinado, pero, en el nivel en el que nos encontramos todavía, no será tan obvio, ya que los problemas no serán tan complicados.

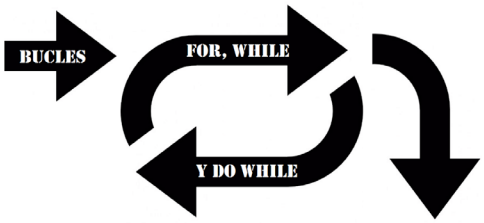

Fig.19. Tipos de bucles.

Vídeo 3. "Utilizando estructuras de control de flujo"

https://vimeo.com/950693238/d2cf75d8c5

/ 11. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema, hemos visto que existe la posibilidad de controlar el flujo de ejecución de un programa haciendo que se ejecuten unos bloques u otros según queramos.

Las sentencias condicionales nos permiten tomar dos posibles caminos: que la condición sea cierta y que la condición sea falsa, pero no ambos a la vez.

Con las sentencias repetitivas conseguimos que un bloque de código se ejecute muchas veces, tantas como indique una cierta condición, sin necesidad de repetir el bloque en código, haciendo mucho más pequeños nuestros programas de esta forma.

Con las sentencias de salto podemos transferir el control de ejecución de un punto a otro del programa, pero son muy peligrosas.

Por último, hemos visto que podemos utilizar los operadores lógicos para combinar condiciones y hacerlas más compactas, aunque más difícil de entender.

A continuación podrás ver el esquema resumen del tema.

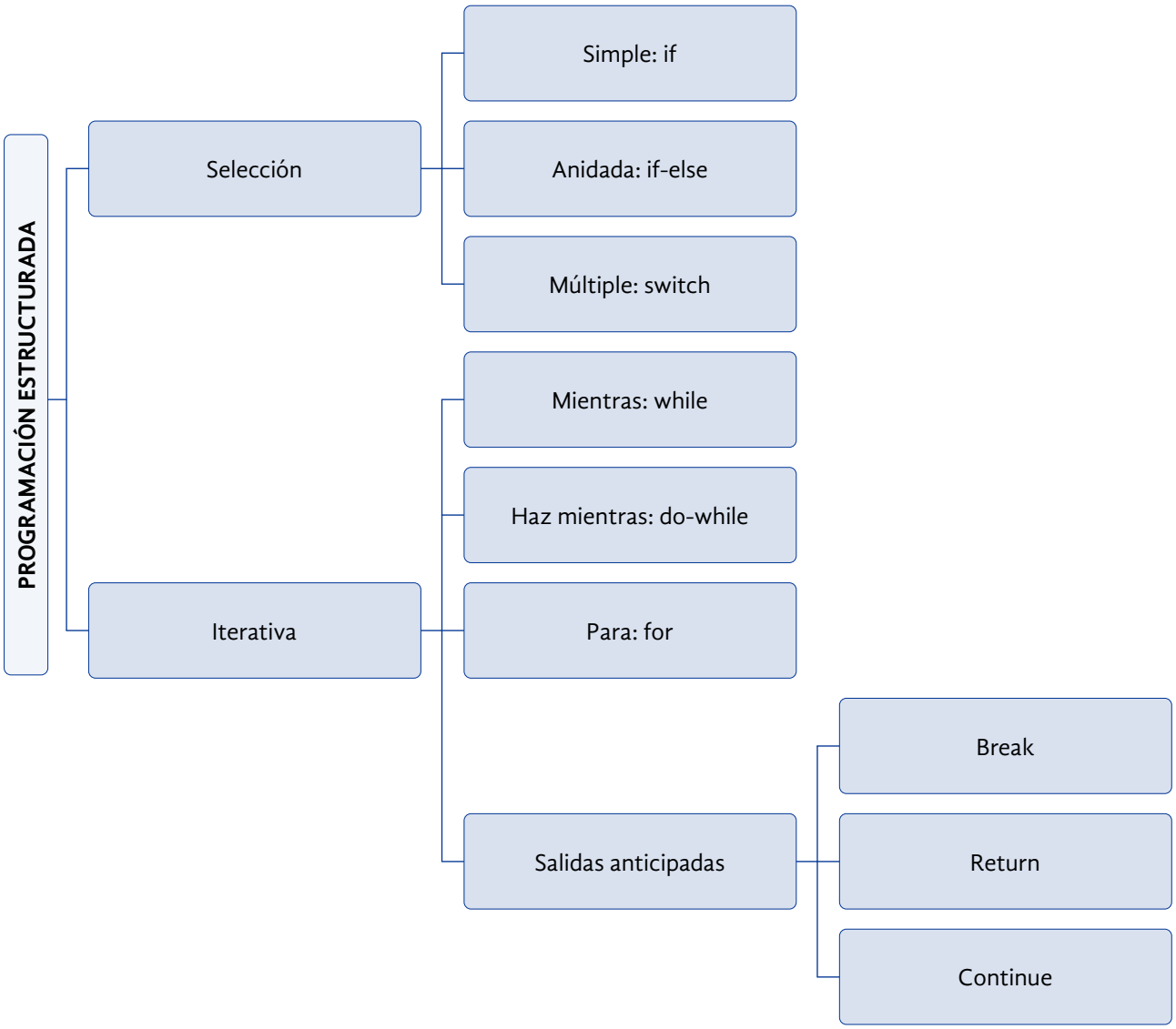

Fig.20. Esquema resumen del tema.

Resolución del caso práctico de la unidad

El pseudocódigo de resolución del algoritmo es el siguiente:

```
Inicio
    Mirar la cartelera
    Si el cine es apto para niños entonces
        Entrar al cine
        Comprar palomitas
    Fin Si
    Sino
        Ir al parque
    Fin sino
Fin
```

Fig.21. Resolución del algoritmo de selección.

/ 12. Bibliografía

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Giani, C. (2024, 9 abril). *Diagrama de flujo. Qué es, tipos, simbología y ejemplos*. Concepto. Recuperado de: https://concepto.de/diagrama-de-flujo/

Programación USM (s. f.). *Sentencias de control*. Recuperado de: http://progra.usm.cl/apunte/materia/sentencias-de-control.html

Wikipedia, The Free Encyclopedia (2024, 16 enero). *Bucle while*. Recuperado de: https://es.wikipedia.org/wiki/Bucle_while

Wikipedia, The Free Encyclopedia (2023, 17 marzo). *Bucle do*. Recuperado de: https://es.wikipedia.org/wiki/Bucle_do

Wikipedia, The Free Encyclopedia (2024, 21 marzo). *Estructuras de control*. Recuperado de: https://es.wikipedia.org/wiki/Estructuras_de_control